If the request is authorized, the next middleware in the pipeline executes as normal. If the request is not authorized, the `AuthorizationMiddleware` short-circuits the middleware pipeline, and the endpoint middleware is never executed.

NOTE The call to `UseAuthorization()` must always be placed after `UseRouting()` and `UseAuthentication()`, but before `UseEndpoints()`. `WebApplication` automatically adds all this middleware in the correct order, but if you override the position in the pipeline, such as by calling `UseRouting()`, you must make sure to maintain this overall order.

The `AuthorizationMiddleware` is responsible for applying authorization requirements and ensuring that only authorized users can execute protected endpoints. In section 24.2.1 you'll learn how to apply the simplest authorization requirement to an endpoint, and in section 24.2.2 you'll see how the framework responds when a user is not authorized to execute an endpoint.

24.2.1 Preventing anonymous users from accessing your application

When you think about authorization, you typically think about checking whether a particular user has permission to execute an endpoint. In ASP.NET Core you normally achieve this by checking whether a user has a given claim.

There's an even more basic level of authorization we haven't considered yet: allowing only authenticated users to execute an endpoint. This is even